Pr. I. HAMZAOUI / Filière : IITE-2
Module : Deep Learning & Reinforcement Learning
Année universitaire 2025/2026

Travaux Pratiques N°8

Objectifs du TP :

- Mettre en pratique un pipeline complet de classification multiclasse d'images à l'aide d'un **réseau de neurones convolutif (CNN)**.
- Prétraiter les images en appliquant le redimensionnement, la normalisation et le découpage en ensembles d'entraînement, de validation et de test.
- Construire, compiler et entraîner un modèle CNN from scratch en intégrant des techniques d'amélioration **(data augmentation, early stopping, ReduceLROnPlateau).**
- Évaluer les performances du modèle à l'aide de métriques adaptées (accuracy, matrice de confusion, rapport de classification et courbes d'apprentissage).
- Implémenter une **approche de Transfer Learning** en utilisant un modèle pré-entraîné et l'adapter à une tâche de classification multiclasse.
- Comparer les performances entre le modèle CNN classique et le modèle basé sur le Transfer Learning.

Livrables à rendre : **Le notebook Google colab complété devra être envoyé à cette adresse mail :** hamzaoui.ikhlasse@ucd.ac.ma

- Le notebook devra être nommé selon le format suivant : TP8_Nom_Prenom_IITE2.ipynb, avec :

- les cellules de code exécutées
- les graphiques générés
- les réponses aux questions.

Dataset :

- Jeu de données utilisé : tf_flowers (TensorFlow Datasets)
- Nombre d'observations : Environ 3670 images réparties sur 5 classes
- Structures de données :
  - ✓ Images RGB (3 canaux)
  - ✓ Tailles variables (non uniformes selon les images)
  - ✓ Format : couples (image, label)

Travail demandé : Partie I : Pipeline de construction d'un CNN pour une classification multiclasse

Q1) Importer les bibliothèques nécessaires, charger le dataset tf_flowers, afficher le nombre d'images et les classes, puis visualiser quelques images du jeu de données.

```python
# Q1) IMPORTATION DES BIBLIOTHÈQUES

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
```

```python
# CHARGEMENT DU DATASET
dataset, info = tfds.load("tf_flowers", with_info=True, as_supervised=True)
# Le dataset est fourni dans un seul ensemble "train"
data = dataset["train"]
print("Nombre total d'images :", info.splits["train"].num_examples)
print("Noms des classes :", info.features["label"].names)

# VISUALISATION DE QUELQUES IMAGES BRUTES
class_names = info.features["label"].names

plt.figure(figsize=(12, 8))
for i, (image, label) in enumerate(data.take(6)):
    plt.subplot(2, 3, i + 1)
    plt.imshow(image)
    plt.title(class_names[label.numpy()])
    plt.axis("off")
plt.tight_layout()
```

Q2) Afficher la shape de quelques images. Que remarquez-vous concernant les dimensions des images ? Pourquoi cette observation est-elle importante avant la phase de prétraitement ?

Q3) Convertir le dataset en deux structures contenant les images et les labels sous forme de tableaux NumPy. Pourquoi cette transformation est-elle nécessaire dans le pipeline de traitement des données ?

Q4) Exécuter ce code pour appliquer un redimensionnement à une taille uniforme (128×128), et une normalisation des valeurs de pixels entre 0 et 1. Vérifier ensuite les dimensions des images obtenues ainsi que les valeurs minimales et maximales.

```python
# 4) REDIMENSIONNEMENT ET NORMALISATION
processed_images = []
for image in images:
    # Redimensionner toutes les images à 128x128
    image_resized = tf.image.resize(image, (128, 128))
    # Convertir en float32 puis normaliser entre 0 et 1
    image_resized = tf.cast(image_resized, tf.float32) / 255.0
    processed_images.append(image_resized.numpy())
processed_images = np.array(processed_images)

print("Shape après prétraitement :", processed_images.shape)
print("Shape d'une image prétraitée :", processed_images[0].shape)
print("Valeur min :", processed_images[0].min())
print("Valeur max :", processed_images[0].max())
```

Pourquoi est-il nécessaire de convertir les images en float32 avant la normalisation ? Que se passerait-il si les images restaient en type entier (int8 ou uint8) ?

Q5) Procéder à un découpage des données en trois ensembles : entraînement (70%), validation (15%) et test (15%). Vérifier ensuite les dimensions des ensembles obtenus.

Q6) Exécuter le code de data augmentation ci-dessous. Expliquer le rôle de chaque transformation appliquée et l'intérêt de cette étape dans l'entraînement d'un modèle CNN.

```
# 6) DATA AUGMENTATION
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])
```

Q7) Exécuter ce code de construction du modèle CNN et afficher son résumé.

- Quel est l'intérêt d'augmenter progressivement le nombre de filtres (32 → 64 → 128) dans le modèle ?
- Quel est le rôle de la couche GlobalAveragePooling2D() ?

```
# 7) CONSTRUCTION DU MODÈLE CNN MULTICLASSE SIMPLE
model = keras.Sequential([
    keras.Input(shape=(128, 128, 3)),
    data_augmentation,

    layers.Conv2D(32, (3, 3), padding="same", activation="relu"),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), padding="same", activation="relu"),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, (3, 3), padding="same", activation="relu"),
    layers.MaxPooling2D((2, 2)),

    layers.GlobalAveragePooling2D(),

    layers.Dense(128, activation="relu"),
    layers.Dropout(0.4),

    layers.Dense(5, activation="softmax")
])
model.summary()
```

Q8) À partir du résumé du modèle :

- ✓ Choisir une couche Conv2D et expliquer en détail comment est calculé son nombre de paramètres.
- ✓ Choisir une couche Dense et expliquer le calcul du nombre de paramètres associés.
- ✓ Expliquer l'évolution du nombre total de paramètres dans le modèle.

Q9) Compiler le modèle et exécuter le code ci-dessous.
Rappeler le rôle du callbacks ReduceLROnPlateau dans le processus d'entraînement.

```python
# 9) COMPILATION DU MODÈLE
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```python
# 11) CALLBACKS : EARLY STOPPING + REDUCE LR

early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=6,
    restore_best_weights=True
)
reduce_lr = keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=3,
    min_lr=1e-6
)
```

Q10) Entraîner le modèle pendant 20 epochs en utilisant les callbacks EarlyStopping et ReduceLROnPlateau. Selon vous, quels sont les éléments de l'architecture ou du pipeline qui peuvent ralentir l'entraînement du modèle ?

Q11) Évaluer le modèle sur le jeu de test en affichant les valeurs de la loss et de l'accuracy.

Q12) Exécuter le code ci-dessus permettant d'effectuer des prédictions sur le jeu de test et d'afficher quelques exemples réels et prédits.

```python
# 12) PRÉDICTION SUR TOUT LE JEU DE TEST
y_pred_prob = model.predict(X_test)
y_pred = np.argmax(y_pred_prob, axis=1)
print("5 premières classes prédites :", y_pred[:5])
print("5 premières classes réelles  :", y_test[:5])
```

```python
# AFFICHAGE DE QUELQUES EXEMPLES RÉEL / PRÉDIT
plt.figure(figsize=(12, 8))
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(X_test[i])
    plt.title(f"Réel : {class_names[y_test[i]]}\nPrédit : {class_names[y_pred[i]]}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

Répondre aux questions suivantes :

1) Quelle est la différence entre l'évaluation (evaluate) du modèle et la prédiction (predict) ?
2) Que remarquez-vous en comparant les classes réelles et les classes prédites sur les images affichées ?
3) Le modèle est-il toujours correct ? Justifier votre réponse à partir des exemples affichés.

Q13) Exécuter le code ci-dessus afin de générer la matrice de confusion et le rapport de classification.

```python
# 13) MATRICE DE CONFUSION
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(7, 6))
sns.heatmap(
    cm, annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names )
plt.xlabel("Classe prédite")
plt.ylabel("Classe réelle")
plt.title("Matrice de confusion - CNN multiclasse sur tf_flowers")
plt.show()
print("Matrice de confusion :")
print(cm)
```

```python
# RAPPORT DE CLASSIFICATION
print("Rapport de classification :\n")
print(classification_report(y_test, y_pred, target_names=class_names))
report_df = pd.DataFrame(
    classification_report(
        y_test,
        y_pred,
        target_names=class_names,
        output_dict=True
    )
).transpose()
print(report_df.round(3))
```

1) Comment interpréter les valeurs situées sur la diagonale de la matrice ?
2) Identifier les faux positifs et les faux négatifs d'une classe de votre choix ?
3) Quelle classe semble être la mieux prédite ? Et laquelle est la plus difficile à classifier ?

Partie II : Transfer Learning

Q1) À l'aide d'une recherche, expliquer le principe du Transfer Learning. Pourquoi cette approche est-elle particulièrement utile dans les problèmes de classification d'images ?
Q2) Exécuter le code ci-dessous.

```python
# Charger MobileNetV2 sans la partie classification
base_model = keras.applications.MobileNetV2(
    input_shape=(128, 128, 3),
    include_top=False,
    weights="imagenet")
```

```
Downloading data from https://storage.googleapis.com/tensor
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```python
# Gener les couches: on garde les features déjà apprises
for layer in base_model.layers:
    layer.trainable = False
```

```python
# On ajoute une tête de classification
model_tl = keras.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(5, activation="softmax")
])
```

```python
model_tl.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```python
history_tl = model_tl.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=15,
    batch_size=32,
    callbacks=[early_stopping]
)
```

```python
test_loss_tl, test_acc_tl = model_tl.evaluate(X_test, y_test)

print("Accuracy Transfer Learning :", test_acc_tl)
```

```python
print("CNN from scratch :", test_accuracy)
print("Transfer Learning :", test_acc_tl)
```

```python
y_pred_prob_tl = model_tl.predict(X_test)
y_pred_tl = np.argmax(y_pred_prob_tl, axis=1)

print("5 premières classes prédites :", y_pred_tl[:5])
print("5 premières classes réelles  :", y_test[:5])
```

```python
# AFFICHAGE DE QUELQUES EXEMPLES RÉEL / PRÉDIT
plt.figure(figsize=(12, 8))

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(X_test[i])
    plt.title(f"Réel : {class_names[y_test[i]]}\nPrédit : {class_names[y_pred_tl[i]]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Q3) Répondre aux questions suivantes :

1) Effectuer une courte recherche sur l'architecture MobileNetV2. Sur quel type de dataset ce modèle a-t-il été entraîné ?
2) Pourquoi gèle-t-on les couches du modèle pré-entraîné (trainable = False) ?
3) Pourquoi peut-on utiliser des caractéristiques apprises sur ImageNet pour un autre problème (ici classification de fleurs) ?
4) Pourquoi le modèle basé sur le Transfer Learning donne-t-il de meilleurs résultats que le CNN entraîné from scratch ?
5) En observant les images réelles et prédites, que remarquez-vous par rapport au modèle précédent ?